CMPS 6610 Problem Set 04

Answers

Name: Rob Hartley

Place all written answers from `problemset-04.md` here for easier grading.

- **1d.**

File	Fixed-Length Coding	Huffman Coding	Huffman vs. Fixed-Length
f1.txt	1340	826	0.6164
alice29.txt	1039367	676374	0.6508
asyoulik.txt	876253	606448	0.6921
grammar.lsp	26047	17356	0.6663
fields.c	78050	56206	0.7201

The pattern that emerges is that:
Huffman cost $\leq$ Fixed-Length cost. This is because Huffman coding achieves a lower encoding cost relative to Fixed Length by assigning shorter binary codes to more frequently used characters and longer codes to less frequently used characters. The more uneven the distribution between frequencies of the characters being used in a document, the greater the Huffman cost reduction we'd expect to see. Our samples reflect that, and show a cost reduction range of roughly 28% - 38%.

- **1e.**
When all characters in alphabet $\sum$ have the same frequency, Huffman coding produces either a balanced or near balanced tree.
Let characters $c = |\sum|$ and tree height ceiling $h = \lceil \log_2 c \rceil$. The codeword length for each character in the tree will be either h or $h-1$ with $2^h - c$ characters using $h-1$ bits and $2c - 2^h$ characters using h bits. From this information, we can determine average number of bits per character A_b with this equation:

$A_b = \frac{(2^h - c)(h-1) + (2c - 2^h)(h)}{c}$

$A_b = \frac{2^h h - 2^h - ch + c + 2ch - 2^h h}{c}$

$A_b = \frac{-2^h + c + ch}{c}$

$A_b = h + 1 - \frac{2^h}{c}$

If we have n total number of characters in our document and the frequency of each distinct character is the same, then the Huffman coding cost would be:

$$\boxed{\text{Huffman Cost} = n \cdot \left(h + 1 - \frac{2^h}{c} \right)}$$

This equation applies to all documents where each distinct character occurs with equal frequency. The Huffman coding cost will be consistent across documents with the same total number of characters n and the same number of distinct characters c. However, the coding cost may change if either n or c changes.

- **2a.** Construct a binary min-heap algorithm

This algorithm would be a bottom up algorithm, starting at the bottom right internal node (not the leaves) and working its way up the tree. For each node, the node value is compared to its children. If the parent value isn't already the lowest value, it is swapped with the lowest child value, which becomes the new parent. Because the bottom right internal node will also have the highest index value for an

internal node in the list, and all other index values below it will also represent internal nodes, we then decrement the index and repeat the process until we reach the root node. The min-heap is now built.

```
def sift_down(A, i, n):
    while 2*i + 1 < n:
        left = 2*i + 1
        right = 2*i + 2
        smallest = left

        if right < n and A[right] < A[left]:
            smallest = right

        if A[i] <= A[smallest]:
            break

        A[i], A[smallest] = A[smallest], A[i]
        i = smallest


def build_min_heap(A):
    n = len(A)

    for i in range(n // 2 - 1, -1, -1):
        sift_down(A, i, n)
```

buildMinHeap:
This method is a loop that executes $\lfloor \frac{n}{2} \rfloor$ times, doing constant work each time, excluding the call to siftDown.
$W_{BH}(n) = \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \Theta(1)$
$W_{BH}(n) = \lfloor \frac{n}{2} \rfloor \cdot \Theta(1)$
$W_{BH}(n) \in \Theta(n)$

siftDown:
Each node can move downward by no more than its height h in the tree. Each move costs $\Theta(1)$. So each node requires at most $O(h+1)$ work. There are at most $\lceil \frac{n}{2^{h+1}} \rceil$ nodes at height h. Therefore:

$W_{SD} = O\left(n \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{h+1}{2^{h+1}}\right)$
$\sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{h+1}{2^{h+1}} \leq \sum_{h=0}^{\infty} \frac{h+1}{2^{h+1}}$
$\sum_{h=0}^{\infty} \frac{h+1}{2^{h+1}} = \frac{1}{2^1} + \frac{2}{2^2} + \frac{3}{2^3} + \frac{4}{2^4} + \cdots = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \ldots$
$\left(\frac{1}{2}\right) \cdot \sum_{h=0}^{\infty} \frac{h+1}{2^{h+1}} = \left(\frac{1}{2}\right) \cdot \left(\frac{1}{2^1} + \frac{2}{2^2} + \frac{3}{2^3} + \frac{4}{2^4} + \ldots\right) = \frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \frac{4}{32} + \ldots$
$\sum_{h=0}^{\infty} \frac{h+1}{2^{h+1}} - \left(\frac{1}{2}\right) \cdot \sum_{h=0}^{\infty} \frac{h+1}{2^{h+1}} = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \cdots = \sum_{h=0}^{\infty} (\frac{1}{2})^{(h+1)}$

Infinite Geometric Series
$\sum_{i=0}^{\infty} ar^k = \frac{a}{1-r}, |r| < 1$
$\sum_{h=0}^{\infty} (\frac{1}{2})^{(h+1)}, a = \frac{1}{2}, r = \frac{1}{2}$
$\sum_{h=0}^{\infty} (\frac{1}{2})^{(h+1)} = \frac{1/2}{1-1/2} = 1$

Combine
$\left(\frac{1}{2}\right) \cdot \sum_{h=0}^{\infty} \frac{h+1}{2^{h+1}} = 1$
$\sum_{h=0}^{\infty} \frac{h+1}{2^{h+1}} = 2$
$W_{SD} = O(2n)$
$W_{SD} \in O(n)$

$W_{total} = W_{BH}(n) + W_{SD}$
$W_{total} = \Theta(n) + O(n)$

$\boxed{W_{total} \in \Theta(n)}$

- **2b.** Span of your algorithm
  There is no parallelism in the algorithm. The buildHeap loop executes sequentially, as do all the operations in siftDown. Span = Work.
  $\boxed{S_{total} \in \Theta(n)}$

- **3a.** Greedy algorithm for making change

```
def greedyCoins(N, k):
    coins = []
    remaining = N

    for i in range(k, -1, -1):
        denomination = 2 ** i
        count = remaining // denomination
        remaining = remaining % denomination
        coins.append((denomination, count))

    return coins
```

- **3b.** Optimality proof
  The optimal solution for this problem is to return a combination of the minimum total number of coins that, based on their denomination, will add up to a total of N.
  The denomination we are using are coins that are in denominations of powers of 2. Consider any denomination valued 2^i, with 2^k representing the maximum denomination that can be used to divide the total value N.
  Any 2 coins of denomination 2^i can be replaced by one coin of the next highest denomination: $2^i + 2^i = 2^{i+1}$ Therefore an optimal solution cannot contain any more than 1 coin of any denomination smaller than 2^k.
  The total value of these coins smaller than 2^k would be $\sum_{i=0}^{k-1} 2^i$
  Finite Geometric series formula: $\sum_{i=0}^{n-1} ar^i = a \cdot \frac{1-r^n}{1-r}$
  For $\sum_{i=0}^{k-1} 2^i$ where $a = 1, r = 2$ this gives us $1 \cdot \frac{1-2^k}{1-2} = 2^k - 1$ maximum total value for coins with denominations less than 2^k.
  Consider the division algorithm with remainder r, total coin value N, and number of largest denomination coins q, and largest denomination value d:
  $N = qd + r$ where $q = \lfloor \frac{N}{d} \rfloor$ and $0 \leq r < d$
  This gives us: $N = q2^k + r, 0 \leq r < 2^k$ where $q = \lfloor \frac{N}{2^k} \rfloor$
  This shows that an optimal solution must contain q coins of the largest denomination 2^k, and that the maximum value of the smaller denomination coins will be at most $2^k - 1$. This matches the greedy choice.

  Suppose an optimal solution for N contains the greedy choice of q largest coins, but the solution for the remainder r is not optimal. This would mean that a solution for r that uses less coins exists, and that replacing the original solution for r with this new solution would reduce the total number of coins used to get N. This would contradict the assumption that the original solution was optimal, and would require that the remaining amount also be solved optimally. Therefore the optimal substructure property holds. The greedy algorithm produces the minimum number of coins.

  The greedy algorithm is optimal ■

- **3c.** Work and span of your algorithm
  Each iteration of the sequential algorithm greedyCoins performs one calculation for the denomination value, one integer floor division to get the number of coins, one modulo operation to determine the remainder, and one insertion into the output. Assuming all of these are constant work:
  $W(k) = \sum_{i=0}^{k} \Theta(1)$

$W(k) = (k+1) \cdot \Theta(1)$
$W(k) = \Theta(k+1)$
$W(k) = \Theta(k)$

$\boxed{W(k) \in \Theta(k)}$

There is no parallelism, so span = work.

$\boxed{S(k) \in \Theta(k)}$

- **4a.** Fortuito counter example proof
  Suppose we have denominations $D = 1, 3, 4$ and a goal amount of $N = 6$. The greedy algorithm from Geometrica problem 3 will select coins $4 + 1 + 1 = 6$, which will give us the solution in 3 coins. The optimal solution to this problem is $3 + 3 = 6$, which gives us the solution in two coins. Therefore the greedy algorithm is not always optimal with arbitrary denominations like the ones in Fortuito. ■

- **4b.** Optimal substructure property
  A problem has the optimal substructure property if an optimal solution to the problem contains optimal solutions to its subproblems.

  For an optimal solution to the coin problem, if the optimal solution for amount N contains a coin of denomination d, then the remaining coins must form an optimal solution for $N - d$.

  Suppose we had an optimal solution for N using c coins, including coin d. If we remove coin d, then $c - 1$ remaining coins should be an optimal solution for $N - d$. If this were not the case, and we could then have an optimal solution to $N - d$ with fewer coins than $c - 1$, that would imply that we could then add coin d to this smaller set of coins to get a new optimal solution for N using fewer than c coins. This would contradict the optimality of the original solution for N using c coins.

  Therefore the coin problem has optimal substructure property. ■

- **4c.** Fortuito dynamic programming algorithm

```
def minCoins(N, denominations):
    OPT = [float('inf')] * (N + 1)
    chosen = [None] * (N + 1)

    OPT[0] = 0

    for n in range(1, N + 1):
        for d in denominations:
            if d <= n and OPT[n - d] + 1 < OPT[n]:
                OPT[n] = OPT[n - d] + 1
                chosen[n] = d

    if OPT[N] == float('inf'):
        return None

    coins = []
    remaining = N

    while remaining > 0:
        d = chosen[remaining]
        coins.append(d)
        remaining -= d

    return OPT[N], coins
```

This bottom up dynamic programming algorithm uses the optimal substructure property to solve the coin problem with arbitrary denominations. For each integer amount n between 1 and N, OPT stores the minimum number of coins required to make amount n, with $OPT[0] = 0$ and $OPT[n] = \infty$ indicating no solution with the provided denominations. The corresponding denomination of the coin that produced the best solution to the subproblem at $OPT[n]$ is stored in $chosen[n]$.
After optimal solutions to all the subproblems are tabulated, the correct denominations for the optimal solution to N are selected by the final while loop. The number of coins and a list of the coin denominations in the optimal solution are then returned.

The work of the algorithm is most concentrated in the nested loop at the top. The outer N loop runs N times. The inner *denominations* loop runs $k + 1$ times, doing $\Theta(1)$ work each time.

$W(N,k) = \sum_{n=1}^{N} \cdot \sum_{i=0}^{k} \Theta(1)$
$W(N,k) = \sum_{n=1}^{N} \Theta(k+1)$
$W(N,k) = \Theta(N(k+1))$

$\boxed{W(N,k) \in \Theta(Nk)}$

The algorithm has no parallelism, so span is equal to work.

$\boxed{S(N,k) \in \Theta(Nk)}$

- **5a.** Weighted task selection: Optimal substructure?
A problem has the optimal substructure property if an optimal solution to the problem contains optimal solutions to its subproblems.
Consider a set of tasks S, where each task t has a start time s, end time e, and value v. The goal of weighted task selection is to maximize the sum of the value of non-overlapping tasks selected:
$t_i = (s_i, e_i, v_i)$
$max(\sum_{t_i \in S} v_i)$
First, sort all the tasks by increasing end time e_i. Let $OPT(i)$ represent the maximum value that can be obtained from the first i tasks i.e. $\{1, 2, \ldots, (i-1), i\}$
There are two things that can happen:

$OPT[i]$ can exclude task i i.e. $\{1, 2, \ldots, (i-1)\}$, which would mean that the optimal solution comes from the first $(i-1)$ tasks, leaving us with a total value of $OPT(i-1)$

$OPT[i]$ can include task i i.e. $\{1, 2, \ldots, (i-1), i\}$. In this case, we can't include tasks that overlap with t_i. If t_j is the last task that ends no later than t_i begins and their index values are i and j, respectively, then $j = max\{k < i : e_k \leq s_i\}$ or $j = 0$ if no such task exists. This gives us a total value of $v_i + OPT(j)$

If the optimal solution includes t_i, and the tasks are all sorted by increasing finish time, then all the compatible tasks for the optimal solution must come from the first j tasks. If this selected group of tasks is not optimal, then that would imply that there is a better solution to be had by picking a different combination of tasks, which would contradict the assumption that our original solution was optimal. The same holds true for $OPT(i-1)$ if we choose to exclude t_i.

The recurrence this implies is:

$$\boxed{OPT(i) = \begin{cases} 0 & \text{if } i = 0 \\ \max\{OPT(i-1), v_i + OPT(j)\} & \text{if } i \geq 1 \end{cases}}$$

Therefore, in both cases, an optimal solution to weighted task selection contains an optimal solution to its relevant subproblem, proving that weighted task selection has the optimal substructure property.■

- **5b.** Does the greedy choice property hold?
No, not for selecting weighted task scheduling. The greedy algorithm that selects for earliest finishing time does produce an optimal solution for unweighted task scheduling, but it doesn't guarantee an optimal solution for weighted task scheduling. The greedy algorithm that selects for highest value task

also fails to produce an optimal solution for weighted task scheduling.
Consider these tasks:

Counterexample 1: Earliest Finishing Time

Task	Start	End	Value
A	0	2	2
B	2	4	2
C	0	4	10

Using earliest finishing time, greedy will choose tasks A and B. $2 + 2 = 4$
The optimal solution is to choose task C, giving a value of 10.

Counterexample 2: Highest Value Task

Task	Start	End	Value
A	0	4	9
B	0	2	6
C	2	4	6

Using highest value task, greedy will choose task A for a value of 9.
The optimal solution is to choose tasks B and C. $6 + 6 = 12$.

Neither greedy strategy can guarantee an optimal solution to weighted task scheduling.

- **5c.** Dynamic programming algorithm for Weighted Task Selection

```
from bisect import bisect_right

def weightedTaskSelection(tasks):
    # Sort tasks (work). Each task is a 3-tuple (start, end, value).
    tasks = sorted(tasks, key=lambda t: t[1])
    n = len(tasks)

    # Use 1-based indexing.
    tasks = [None] + tasks

    OPT = [0] * (n + 1)
    chosen = [False] * (n + 1)
    p = [0] * (n + 1)

    # Create the array (work).
    ends = [tasks[i][1] for i in range(1, n + 1)]

    # Find the last compatible preceding task with binary search (work).
    for i in range(1, n + 1):
        p[i] = bisect_right(ends, tasks[i][0], 0, i - 1)

    OPT[0] = 0

    # Build OPT and chosen (work).
    for i in range(1, n + 1):
        exclude = OPT[i - 1]
        include = tasks[i][2] + OPT[p[i]]
```

```
        if include > exclude:
            OPT[i] = include
            chosen[i] = True
        else:
            OPT[i] = exclude

    # Reconstruct the selected tasks (work).
    selected = []
    i = n

    while i > 0:
        if chosen[i]:
            selected.append(tasks[i])
            i = p[i]
        else:
            i -= 1

    selected.reverse()

    # Return total weighted task value, list of selected tasks.
    return OPT[n], selected
```

Work:
Sort tasks: $W(n) \in \Theta(n \log n)$
Create the array: $W(n) \in \Theta(n)$
Find the last compatible preceding task with binary search: $W(n) \in \Theta(n \log n)$
Build OPT and chosen: $W(n) \in \Theta(n)$
Reconstruct the selected tasks: $W(n) \in \Theta(n)$

Total work for weightedTaskSelection:
$W_{wt}(n) = \Theta(n \log n) + \Theta(n) + \Theta(n \log n) + \Theta(n)$
$W_{wt}(n) = 2\Theta(n \log n) + 2\Theta(n)$

$\boxed{W_{wt}(n) \in \Theta(n \log n)}$

Span:
There is no parallelism in the algorithm. Span = work.

$\boxed{S_{wt}(n) \in \Theta(n \log n)}$